

page-1-example-1-miniRSA

October 7, 2026

1 EXAMPLE-1: mini RSA with identity padding in quantum circuit with qiskit

dr. kornpob bhirombhakdi 2026-Oct-6

1. this example demonstrates a quantum circuit implementation (qiskit, python3) of RSA encryption/decryption.
2. modular exponentiation is implemented as a reversible oracle U_k : $|x\rangle|y\rangle \rightarrow |x\rangle|y \text{ XOR } (x^k \bmod N)\rangle$. the same builder is used for both directions: public-key e encrypts ($C = M^e \bmod N$) and private-key d decrypts ($M = C^d \bmod N$).
3. flow: $M(\text{bits}) \rightarrow M(\text{int}) \rightarrow \text{padding} \rightarrow \text{encrypt} \rightarrow \text{serialize} \rightarrow \text{transmit/receive} \rightarrow \text{deserialize} \rightarrow \text{decrypt} \rightarrow \text{depadding} \rightarrow \text{MESSAGE}$. padding is the identity here, so the integer goes straight through.
4. usage: set `testcase = (M_bits, p, q, e)` and run. the message value must be smaller than $N = p \cdot q$, and N must fit in $n = N.\text{bit_length}()$ qubits per register ($2n$ qubits in total).
5. limitations: the oracle is a lookup table. $x^k \bmod N$ is computed classically for every $x < N$ and baked into the circuit as multi-controlled X gates, so gate count grows exponentially with n . this is a demonstration of the circuit structure, not a scalable quantum modular exponentiation (which needs modular adders and multipliers with ancilla qubits). identity padding and tiny primes are also not secure, so this is for learning only.

```
[50]: """
EXAMPLE-1: mini RSA with identity padding
testcase: M=010, p,q=3,5, e=3, (N=15, phi=8, d=3)
flow: M(bits) -> M(int) -> padding -> encrypt -> serialize -> transmit/receive
      ↪ -> deserialize -> decrypt -> depadding -> MESSAGE
"""
# !pip install qiskit qiskit-aer

from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister, transpile
from qiskit_aer import AerSimulator
from math import gcd, isqrt

# ---- reversible oracle: |x>|y> -> |x>|y XOR (x^k mod N)>, x = 0..N-1 ----
# k is an integer key (k = e to encrypt, k = d to decrypt), not a symbolic
      ↪ exponent
def modexp_oracle(qc, src, dst, k, n, N):
    for x in range(N):
```

```

        out = pow(x, k, N)
        for bit in range(n):
            if (out >> bit) & 1:
                qc.mcx(list(src), dst[bit], ctrl_state=x)

def load_integer(qc, reg, value):
    """set register to basis state |value> (qubit i = bit i of value)"""
    for i in range(len(reg)):
        if (value >> i) & 1:
            # note operator `value >> i` shifts value in bin-expression by
            ↪ i-bin-digit to the right
            # example: if value = 0110101
            # value >> 1 --> 011010[:1 dropped]
            # value >> 2 --> 01101[:01 dropped]
            qc.x(reg[i]) # circuit is modified in-place

def init_circuit(n):
    """registers + empty circuit (n input qubits, n output qubits, n classical
    ↪ bits)"""
    a = QuantumRegister(n, "in")
    b = QuantumRegister(n, "out")
    c = ClassicalRegister(n, "res")
    qc = QuantumCircuit(a, b, c)
    return qc, a, b, c

def run_oracle(value, k, n, N):
    """load |value>, apply U_k, measure output register, return integer"""
    # value,k = (M_padded, e) for encryption, (C, d) for decryption
    qc, a, b, c = init_circuit(n)
    qc.barrier()
    load_integer(qc, a, value)
    qc.barrier()
    modexp_oracle(qc, a, b, k, n, N)
    qc.barrier()
    qc.measure(b, c) # arguments measure(qubit, cbit)
    sim = AerSimulator()
    counts = sim.run(transpile(qc, sim), shots=100).result().get_counts()
    return int(max(counts, key=counts.get), 2), qc

```

```

[83]: ##### testcase = (M_bits,p,q,e)
testcase = ("010",3,5,3)
# testcase = ("101",3,5,3)
# testcase = ("011",3,7,5)
# testcase = ("011",3,11,3)
# testcase = ("011",11,13,7)
# testcase = ("01101",11,13,7)
# testcase = ("011011",11,13,7)

```

```
# testcase = ("0110101",11,13,7)
# testcase = ("01100101",11,13,7)
# testcase = ("011001011",11,13,7) # expect fail
```

```
[84]: ##### stress test
# testcase = ("011001011", 11, 19, 7) # N=209, n=8, 16 qubits (done)
↳ ~1second
# need M < N; use int(M)=N-1 for worst case:
# testcase = ("0110101110", 23, 29, 3) # N=667, n=10, 20 qubits (e=3:
↳ gcd(3,616)=1) ~4seconds
# testcase = ("10001111010", 31, 37, 7) # M=1146=N-1, N=1147, n=11, 22
↳ qubits (~1 min) ~20seconds
# testcase = ("0111111111", 61, 67, 7) # M=1023, N=4087, n=12, 24 qubits
↳ (slow, minutes) ~247seconds
```

```
[85]: ##### edge cases
# testcase = ("11111101", 11, 23, 7) # expect fail # N=253, just under 2^8:
↳ register almost fully used, M=253? no: 11111101=253 > N-1 -> expect FAIL
↳ (value >= N? 253 >= 253)
# testcase = ("11111100", 11, 23, 7) # expect pass pair # M=252=N-1, N=253,
↳ n=8
```

```
[86]: from datetime import datetime, timezone

M_bits,p,q,e = testcase
local_now = datetime.now()

# ---- key setup (classical) ----
N, phi = p * q, (p - 1) * (q - 1)
d = pow(e, -1, phi)
m = len(M_bits) # message bits
n = N.bit_length() # qubits per register = ceil(log2 N) = 4

#####
#####

def is_prime(x):
    return x > 1 and all(x % i for i in range(2, isqrt(x) + 1))

assert is_prime(p) and is_prime(q) # p, q prime
assert p < q # distinct, so phi = (p-1)(q-1) is
↳ valid
assert p % 2 == 1 and q % 2 == 1 # odd primes
assert 1 < e < phi and gcd(e, phi) == 1 # e invertible mod phi
assert (d * e) % phi == 1 # d is the inverse of e
# assert 2**m <= N # message fits RSA range
```

```

assert N <= 2**n                                # register holds 0..N-1
assert all(pow(pow(M, e, N), d, N) == M for M in range(N)) # round trip,
↳ every M

#####
#####

# ---- SENDER ----
# M_bits = "010"
M_int = int(M_bits, 2)                          # 2
M_padded = M_int                               # identity padding
C, qc_enc = run_oracle(M_padded, e, n, N)        # quantum:  $C = M^e \bmod N$ 
C_bits = format(C, f"0{n}b")                   # serialize, fixed width n
print(f"sender : {M_bits} -> {M_int} -> {M_padded} -> {C} -> {C_bits}")

# ---- transmit / receive ----
received = C_bits

# ---- RECEIVER ----
C_rx = int(received, 2)                        # deserialize
M_padded2, qc_dec = run_oracle(C_rx, d, n, N)   # quantum:  $M = C^d \bmod N$ 
M_int2 = M_padded2                             # depad (identity)
M_bits2 = format(M_int2, f"0{m}b")
print(f"receiver: {received} -> {C_rx} -> {M_padded2} -> {M_int2} -> {M_bits2}")

assert C == pow(M_int, e, N)
assert C_bits == format(pow(M_int, e, N), f"0{n}b")
assert M_bits2 == M_bits

print("OK: round trip recovers", M_bits2)
print(qc_enc.draw(fold=120).__str__()[0])      # circuit available as qc_enc /
↳ qc_dec

local_done = datetime.now()
print(f"time-use: {(local_done - local_now).total_seconds():.3f} seconds")

```

```

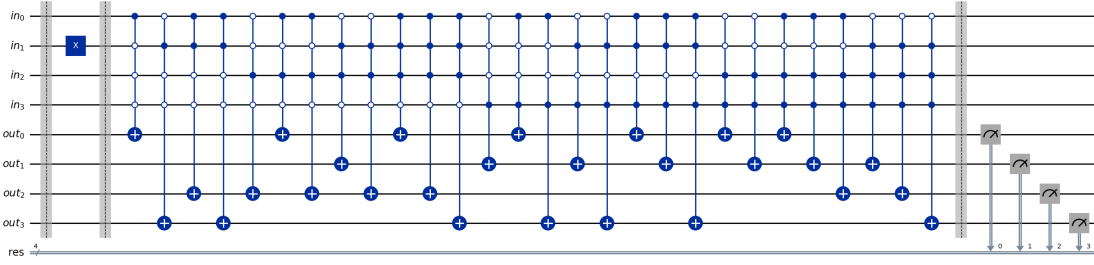
sender : 010 -> 2 -> 2 -> 8 -> 1000
receiver: 1000 -> 8 -> 2 -> 2 -> 010
OK: round trip recovers 010

```

```
time-use: 0.167 seconds
```

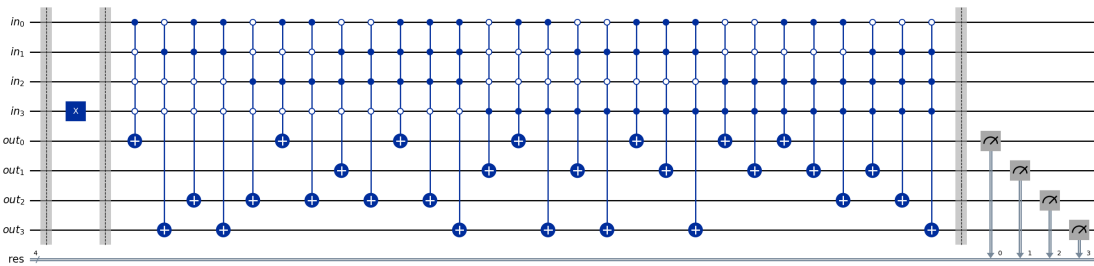
```
[87]: qc_enc.draw('mpl', fold=-1, filename='example-1-rsa-encrypt-circuit.png')
```

[87]:



```
[88]: qc_dec.draw('mpl', fold=-1, filename='example-1-rsa-decrypt-circuit.png')
```

[88]:



1.1 Theory/Code Explained

1.1.1 1. RSA in four lines

- key setup: pick primes $p < q$, set $N = p \cdot q$ and $\phi = (p-1)(q-1)$. choose e with $\gcd(e, \phi) = 1$, then $d = e^{-1} \bmod \phi$.
- encrypt: $C = M^e \bmod N$
- decrypt: $M = C^d \bmod N$
- why it works: $e \cdot d = 1 \pmod{\phi}$, so $M^{(e \cdot d)} = M \pmod{N}$ for every $0 \leq M < N$. the code verifies this by brute force over all M in $\text{range}(N)$.

worked example (this notebook): $p, q = 3, 5$, $N = 15$, $\phi = 8$, $e = 3$, $d = 3$. message $M = 2$ gives $C = 2^3 \bmod 15 = 8$, and decrypting gives $8^3 \bmod 15 = 512 \bmod 15 = 2$.

1.1.2 2. From a function to a circuit

a quantum circuit can only apply reversible (unitary) operations, so $x \rightarrow x^k \bmod N$ cannot be applied directly (it is not one-to-one in general). the standard fix is an oracle that keeps the input and XORs the result into a second register:

$U_k : |x\rangle|y\rangle \rightarrow |x\rangle|y \oplus (x^k \bmod N)\rangle$

- the output register starts at $|y=0\rangle$, so $0 \oplus f(x) = f(x)$.
- the input register $|x\rangle$ passes through unchanged, which makes the map a permutation of basis states (hence unitary).
- the same builder serves both directions: $k = e$ encrypts, $k = d$ decrypts.

1.1.3 3. How the oracle is built (lookup table)

`modexp_oracle` loops over every x in $0..N-1$: 1. compute $\text{out} = \text{pow}(x, k, N) = x^k \bmod N$ classically. 2. for each bit that is 1 in `out`, add a multi-controlled X (`mcx`) targeting that output qubit. 3. the controls are all n input qubits, with `ctrl_state=x`, so the gate fires only when the input register equals the integer x .

so the circuit is a truth table written in gates. at most one x matches, so exactly the bits of $x^k \bmod N$ get flipped in the output register.

1.1.4 4. Registers and bit order

- `n = N.bit_length()` qubits per register (input `in`, output `out`) plus `n` classical bits `res`, so $2n$ qubits in total.
- `load_integer` sets the input register to $|value\rangle$ with qubit i holding bit i of the value (little-endian), using one X per 1-bit.
- qiskit prints measurement strings with bit 0 on the right, so `int(bitstring, 2)` reads the result back directly.

1.1.5 5. Code map

step	code	classical / quantum
key setup, validation	<code>p, q, e -> N, phi, d, n</code> and the asserts	classical
bits to integer, padding	<code>int(M_bits, 2)</code> , identity pad	classical
encrypt	<code>run_oracle(M_padded, e, n, N)</code>	quantum (simulated)
serialize	<code>format(C, f"0{n}b")</code> , fixed width <code>n</code>	classical
transmit / receive	<code>received = C_bits</code>	classical
decrypt	<code>run_oracle(C_rx, d, n, N)</code>	quantum (simulated)
depad, bits	identity depad, <code>format(M_int2, f"0{m}b")</code>	classical

`run_oracle` loads the input, applies U_k , measures the output register, and returns the integer. the input is a basis state and the oracle is deterministic, so the result is the same on every shot.

1.1.6 6. Constraints to remember

- the message value must be smaller than N (`int(M_bits, 2) < N`). a larger value is outside the oracle's domain, encrypts to the wrong thing, and does not round-trip.
- `p, q` must be distinct odd primes, and $1 < e < \text{phi}$ with $\text{gcd}(e, \text{phi}) = 1$.
- the message length `m` can differ from `n`: the plaintext is formatted to `m` bits and the ciphertext to `n` bits.

1.1.7 7. What is (and isn't) quantum here

- the quantum part is only the modular exponentiation, and it is simulated on `AerSimulator` rather than run on hardware.
- the lookup table is computed classically with `pow(x, k, N)` for every x , so the circuit is not a quantum speedup. gate count grows roughly as $N * n$, exponential in qubit count, so this is practical only for small N (a few qubits per register).

- a scalable version would build modular exponentiation from arithmetic circuits (modular adders and multipliers with ancilla qubits), as in Shor's algorithm.
- identity padding and tiny primes are not secure. this is a teaching example of circuit structure.

[]: